

Pipeline Progress Report

March 2026

Overview

This report summarizes completed engineering work and the remaining high-priority items across the compliance pipeline. Broadly, the work to date has focused on four areas: pipeline consolidation, rule extraction quality, Lean generation hardening, and repository hygiene. At present, the completed markdown work documents the architectural and implementation progress already finished, while the remaining pasted notes identify the main unresolved bottlenecks and the recommended next steps.

1 Completed Work

1.1 Pipeline Consolidation (Phases 0–4)

The pipeline has been substantially consolidated and stabilized.

- Shared utilities were centralized into `scripts/utils/__init__.py`, including helpers for JSONL read/write, JSON block extraction, numeric and boolean parsing, question-type normalization, base type inference, and issuer schema construction.
- Multiple scripts were updated to import from `scripts.utils`, while preserving direct execution through `try/except ImportError` fallbacks.
- In `schema_reconcile.py`, sub-pipeline stages now call `rule_anchored_extract` and `type_infer` through direct function entry points instead of mutating `sys.argv`.
- In `type_infer.py`, reconciliation report rows now use the true `provisional_type` per field instead of a hardcoded placeholder.
- In `rule_anchored_extract.py`, table row matching was improved using overrides, dynamic keywords from field names, and token-overlap fallback so extraction is no longer limited to a tiny fixed mapping.
- In `llm_generate_lean.py`, indentation and merge logic were corrected, and the reconciled schema is now the default issuer-schema input.

1.2 Legacy Cleanup and Evidence-First Flow

- Superseded scripts were moved to `scripts/archive/`, with documentation explaining why they were retired.

- The default issuer schema path for Lean generation now points to the reconciled schema in `data/processed/reconcile_run_v3/issuer_schema_reconciled.json`, reinforcing an evidence-first workflow in which reconciliation precedes Lean generation.
- `extract_lean_to_json.py` was converted into a reusable library-style module, and Lean generation now always writes JSON output through `-json-out`, eliminating a previous manual step.

1.3 Lean Pipeline Integration

- `generate()` entry points were added to `gen_core_from_issuer_schema.py`, `gen_rules_lean.py`, and `gen_main_lean.py`.
- `verify_one.py` now calls these generators in-process rather than spawning three Python sub-processes.
- The extra `-` before `lake exe compliance` arguments was removed so that `-in` and `-out` correctly reach the executable.
- `gen_core_from_issuer_schema.py` now uses correct defaults for `Option` types, avoiding double-wrapping and using sensible `.getD` defaults.
- `gen_rules_lean.py` was hardened through safer `generatedRuleset` extraction, incomplete `if-then` repair in `failReason`, expanded unsafe-expression detection and stubs, and Lean 4 method rewrites such as `.getD` and `==` in list lambdas.
- Windows-safe logging was introduced by removing problematic Unicode characters that had been breaking `cp1252` consoles.

1.4 Rule Extraction Improvements

Three core extraction issues were addressed: missing regulations, misattributed `rule_ids`, and weak `maps_to` to quality.

Regulation Anchoring. A deterministic anchoring mechanism was added.

- `pre_identify_regulations(page_text)` detects top-level headings such as `N.` or `8A.` using regex with footnote guards.
- `validate_reg_anchoring(rule, visible_regs)` performs a soft validation check: if the extracted `rule_id` is not visible on the page, confidence is reduced and repair notes are added rather than hard-dropping the rule.
- Visible regulation numbers are injected into the extraction prompt.
- `strip_page_numbers(page_text)` removes bare page numbers at the top of pages so they do not interfere with anchoring logic.

CoT and Few-Shot for `maps_to`.

- The system prompt was extended with explicit `MAPS_TO REASONING` steps.
- A second built-in few-shot example was added for Regulation 7(1)(b), mapping depository agreement requirements to `has_depository_agreement : Bool`.

Two-Pass Extraction.

- Pass 1 now performs lightweight regulation identification with `reg_number`, `clause_text`, `span_hint`, and `is_proviso`.
- Regex-detected parent regulation context is injected so sub-clauses are emitted with full prefixes such as 6(3)(ii) instead of bare (3)(ii).
- Pass 2 extracts one clause at a time using pre-validated `RULE_ID` and `LEAN_ID`.
- A `-no-two-pass` option preserves the legacy single-window path for A/B comparison.

Parser Robustness.

- `coerce_rules_from_parsed()` now wraps a top-level bare dictionary with `rule_id` or `reg_number` into a one-element list, fixing both Pass 1 and Pass 2 cases where a single object is returned instead of an array.

1.5 Data Layout Hygiene and Repository State

- Historical and superseded artifacts were moved under `data/processed/archive/`.
- The active processed-data directory was reduced to the current chain of relevant outputs, including the judged rules, reconciled schema, issuer facts, compliance report, generated Lean, and the latest debug rules file.
- Consolidation work was committed and pushed to the main branch.

2 Current Status by Work Stream

2.1 1. Rule Extraction

Rule extraction is in good shape overall. The two-pass extraction framework is now working, and the remaining issues are mainly polish items rather than foundational blockers. The immediate unblocker is the `coerce_rules_from_parsed` fix for the "clauses" key. After that, the outstanding issues are relatively minor: occasional dropped short-text clauses such as 5(1)(d), sub-clause parent prefixing edge cases, and judge JSON parsing cleanup.

2.2 2. RHP Data Extraction

This remains the main bottleneck. Compared with rule extraction, substantially more work is still required here.

- **Problem A: Table evidence is still too narrow.** The current `find_table_row()` logic in `rule_anchored_extract.py` is effectively hardcoded to a very small set of fields, meaning most fields silently bypass table evidence. The most important next step is to replace this with fuzzy token matching against row labels so the approach generalizes across fields.
- **Problem B: Financial-year header detection is too narrow.** Although `pdf_tables.py` already detects financial-year headers, it misses common RHP table formats such as "For the financial year ended March 31, 2025." This can be fixed with a broader regex that also recognizes variants such as `March 31, YYYY`, `31st March, YYYY`, and fiscal-year shorthand forms such as `2024-25`.

- **Problem C: Regulation cross-references in the RHP are underused.** RHPs often explicitly mention phrases like “Regulation X of the SEBI ICDR Regulations.” These are high-value signals that can be used to boost page selection, evidence scoring, and extraction confidence for the corresponding rules.
- **Problem D: Eligibility sections need explicit prioritization.** Sections titled “Basis for the Offer Price,” “Eligibility,” or “Compliance with Regulation 6” are near-standard and likely contain the highest-value extraction targets. These sections should be prioritized in page selection.

2.3 3. Lean Code Generation and Verification

The generator scripts themselves are largely structurally sound. The remaining problem is that LLM-generated Lean predicates often contain syntax or typing errors that are not automatically repaired. Several planned improvements are still pending:

- APOLLO repair loop.
- Lean compiler error feedback into the LLM.
- Generator/corrector prompt split.

Among these, the APOLLO repair loop is the most important missing piece, because it is expected to raise Lean compilation success materially beyond the current low baseline.

3 Remaining Priority Tasks

The main tasks still left to do are summarized below in recommended order.

1. Finish the current extraction run and review the resulting v4 outputs.
2. Replace hardcoded table-row matching with fuzzy matching in `find_table_row()`.
3. Expand `FY_HEADER_RE` in `pdf_tables.py` to capture March 31-style date headers and fiscal-year variants.
4. Add ICDR regulation cross-reference detection in `rule_anchored_extract.py` and use it to boost evidence-page selection and extraction confidence.
5. Add section-priority heuristics for eligibility-focused pages.
6. Implement the APOLLO repair loop for Lean generation and verification.
7. Add Lean compiler feedback and then separate the generator and corrector prompts.